Classificacao da Idade de Edificios a partir de Imagens de Satelite

Hack4Dev - Guia Tecnico Integral (Notebooks 1 a 4) | Transfer Learning Madrid -> Amsterdam

Pagina 1 - Os 10 Passos para Resolver o Desafio

O desafio consiste em classificar a **idade de construcao de edificios** (4 classes ordinais) a partir de series temporais de refletancia espectral de satelite Landsat (1984-2025), treinando em **Madrid** (cidade rotulada) e transferindo o modelo para **Amsterdam** (cidade-alvo, com poucos rotulos disponiveis). Abaixo esta o roteiro completo, passo a passo, tal como definido nos 4 notebooks do hackathon.

1. Compreender o problema e os rotulos

O alvo (y) e a **classe de idade** (1 a 4), atribuida a partir de weighted_mean_year (ano medio de construcao ponderado por area). As fronteiras das classes 3 e 4 sao fixas (1984-2004 e 2004-2024, o periodo coberto pelo satelite); as fronteiras das classes 1 e 2 sao especificas por cidade (Madrid: evento em 1960 - Planos de Desenvolvimento; Amsterdam: evento em 1945 - fim da ocupacao).

2. Carregar os dados brutos (Notebook 2)

Ler os parquets madrid_train.parquet e amsterdam_data.parquet. Formato largo: 1 linha = 1 pixel de 30x30 m x 1 ano, com ate 3 observacoes Landsat por linha (obs_1/2/3).

3. Derivar a classe de idade (age_class)

Aplicar pd.cut() sobre weighted_mean_year com as fronteiras especificas de cada cidade, gerando o rotulo inteiro 1-4 que servira de y.

4. Achatar (flatten) para 1 observacao valida por ano

Para cada banda espectral, escolher a primeira observacao com qa_valid=True entre os 3 slots. Remover linhas sem nenhuma observacao valida e remover contaminacao atmosferica (Blue > 15.000).

5. Preencher lacunas temporais (gap filling)

Construir uma grelha completa (pixel x ano) e preencher os anos em falta: interpolacao linear para lacunas internas; preenchimento pelo valor mais proximo nas bordas (antes da 1a e depois da ultima observacao).

6. Calcular indices espectrais derivados

A partir das 6 bandas base, calcular 5 indices: NDVI, NDBI, UI, MNDWI e BSI - combinacoes normalizadas que realcam vegetacao, area construida e solo exposto.

7. Engenharia de atributos por pixel (feature engineering)

Colapsar cada serie temporal de ate 40 anos num unico vetor de 60 estatisticas por pixel: medias/desvios globais, do periodo inicial (1984-2003), do periodo tardio (2004+), variacao ano-a-ano, e 2 indicadores de cobertura temporal.

8. Padronizar (StandardScaler) e treinar o modelo base em Madrid

Ajustar o StandardScaler apenas em Madrid (nunca em Amsterdam, para evitar vazamento de dados). Treinar um RandomForestClassifier e validar com validacao cruzada repetida (5-fold x 5 repeticoes) para obter a metrica macro-F1 com media e desvio-padrao.

9. Medir a transferencia para Amsterdam: zero-shot e few-shot

Aplicar o modelo de Madrid diretamente em Amsterdam sem qualquer rotulo local (zero-shot) para medir o "gap" entre cidades. Depois, usar um pequeno numero de pixels rotulados de Amsterdam (5, 10, 25, 50, 100, 200 por classe) para calcular **protototipos de classe** (vetor medio de atributos) e classificar por proximidade (few-shot prototype transfer, uma tecnica de aprendizagem metrica / metric learning).

10. Avaliar, reportar e entregar

Reportar accuracy, macro-F1, matrizes de confusao e mapas espaciais de erro/incerteza para Madrid (CV), Amsterdam zero-shot e Amsterdam few-shot (para cada orcamento de rotulos). Entregaveis finais: modelo treinado, codigo de adaptacao parametrizavel, apresentacao em slides com grafico F1 vs log2(n amostras), e uma justificativa tecnica escrita (max. 300 palavras).

Paginas 2-3 - Notebook "2-Reading_Data": Entendendo os Dados

2.1 - Origem e formato dos dados

Os dados vem de um pipeline anterior que ja ligou imagens brutas Landsat (5 TM, 7 ETM+, 8 OLI, 9 OLI-2) a registos de construcao de edificios. Cada ficheiro Parquet contem, para uma cidade, uma linha por **combinacao (pixel geografico de 30x30 m, ano)**, cobrindo 1984-2025. Como ate 3 cenas Landsat de verao podem existir no mesmo ano (ranqueadas por nuvens/pixels em falta), cada linha guarda ate 3 conjuntos de observacao lado a lado: obs_1, obs_2, obs_3.

Volumetria real: Madrid - 3.172.765 linhas x 48 colunas, 76.263 pixels unicos, 42 anos distintos (1984-2025). Amsterdam - dataset de transferencia com 25.992 pixels unicos apos limpeza. Os dados ja vem **pre-filtrados**: apenas pixels com >=15% de cobertura de edificios (coluna coverage) e anos de construcao >=1850 sao incluidos.

2.2 - Dicionario de colunas (formato largo original)

Coluna	Tipo	Descricao
pixel_id	str	Identificador unico: {cidade}_{px_key}_{py_key}_{ano}
city	str	"Madrid" ou "Amsterdam"
year	int	Ano de aquisicao da cena
px_key, py_key	int	Indices de grelha UTM (30 m), estaveis entre cenas
pixel_x, pixel_y	float	Coordenadas do centro do pixel em UTM (metros)
coverage	float	Fracao da area do pixel sobreposta por pegadas de edificios (>=0,15)
weighted_mean_year	float	Ano medio de construcao, ponderado pela area dos edificios
Blue_N ... SWIR2_N	float	Refletancia espectral da observacao N (N=1,2,3), bandas Blue/Green/Red/NIR/SWIR1/SWIR2
qa_valid_N	bool	Verdadeiro se a observacao N e um pixel limpo (sem nuvem/sombra/preenchimento)
doy_N	int	Dia do ano da observacao N
scene_id_N / dataset_N	str	Identificador da cena Landsat e colecao (TM vs OLI)
age_class (derivada)	int 1-4	Classe de idade, calculada a partir de weighted_mean_year (ver 2.3)

2.3 - As 4 classes de idade (rotulo y)

weighted_mean_year e continua; nao vem uma classe pre-atribuida no ficheiro. A classe e obtida com pd.cut(weighted_mean_year, bins=[-inf, evento_cidade, 1984, 2004, inf], labels=[1,2,3,4]). As classes 3 e 4 usam as mesmas fronteiras nas duas cidades (porque correspondem ao periodo coberto pelo satelite); as classes 1 e 2 usam um evento historico especifico de cada cidade:

Classe	Madrid	Amsterdam	Contexto
1	Antes de 1960	Antes de 1945	Construcao pre-moderna, anterior ao evento urbanistico local
2	1960 - 1984	1945 - 1984	Construcao pre-satelite, posterior ao evento local
3	1984 - 2004	1984 - 2004	1o periodo coberto pelo registo Landsat
4	2004 - 2024	2004 - 2024	2o periodo coberto pelo registo Landsat (mais recente)

Desequilibrio de classes: a Classe 1 (edificios mais antigos) domina em ambas as cidades, porque a maior parte do parque imobiliario existente e pre-moderno. Em nivel de linha, Madrid tem: Classe 1 = 576.190, Classe 2 = 1.125.456, Classe 3 = 780.886, Classe 4 = 690.233. Este desequilibrio precisa de ser tratado no treino (ex.: class_weight='balanced').

2.4 - O que e feito aos dados antes do treino (nesta fase exploratoria)

- **Flatten (achatamento por ano):** de cada linha (com ate 3 observacoes), escolher a primeira observacao com qa_valid=True para cada banda, via a funcao get_best_obs_band(); descartar linhas sem nenhuma observacao valida em alguma banda.

- **Remocao de outliers atmosfericos:** cenas Landsat 7 em Amsterdam contem neblina nao detetada pela flag QA padrao (ex.: cena de 29/05/2003 com Blue ate 64.126). Regra aplicada: remover linhas com Blue > 15.000 (o valor urbano normal ronda 10.000). Isto removeu 34.845 linhas em Madrid (1,1%) e 12.917 em Amsterdam.
- **Verificacao de qualidade:** confirmar ausencia de pixels corruptos (valor 0 ou saturado 65.535) - restaram apenas 23 linhas corruptas em Madrid e 7 em Amsterdam, marginais.
- **Exploracao de correlacao entre bandas:** Blue/Green/Red sao fortemente correlacionadas entre si (redundancia no visivel); NIR, SWIR1 e SWIR2 trazem informacao adicional - o que justifica usar todas as 6 bandas em vez de apenas RGB.
- **Exploracao visual:** series temporais de pixels individuais, medias anuais por classe (+/- desvio-padrao), e graficos de dispersao media-vs-desvio, que sugerem que a refletancia media por pixel e a variabilidade ano-a-ano carregam sinal discriminativo entre classes.

Nota importante: *nesta fase (Notebook 2) nada ainda e agregado por pixel - os dados continuam em formato longo (1 linha = 1 pixel-ano). A agregacao para 1 linha por pixel so acontece no Notebook 3 (Preprocessing), descrito nas paginas seguintes.*

Pagina 4 - O que e Treinado: Definicao de X e Y

4.1 - A variavel alvo (y)

y = age_class, um inteiro em {1, 2, 3, 4}, um valor por **pixel** (nao por pixel-ano). E um problema de **classificacao multiclasse** - nao de regressao. O modelo nunca preve diretamente um ano de construcao continuo; preve a qual dos 4 intervalos de idade o pixel provavelmente pertence.

4.2 - A matriz de atributos (X)

X e uma matriz com **1 linha por pixel** e **60 colunas (features)**, todas derivadas da serie temporal de 40 anos daquele pixel, ja resumida estatisticamente (ver pagina 5). Apos o pre-processamento: X_madrid tem forma (76.263 x 60) e X_amsterdam tem forma (25.992 x 60).

Grupo de atributos	N features	O que captura
Media geral por banda/indice (6 bandas + 5 indices)	11	Nivel medio de refletancia ao longo de todo o periodo
Desvio-padrao geral por banda/indice	11	Variabilidade geral da serie temporal
Media periodo inicial (1984-2003)	6	Uso do solo pre-existente / o que estava no local antes
Desvio-padrao periodo inicial	6	Variabilidade no periodo inicial
Media periodo tardio (2004+)	6	Sinal espectral do edificio atual
Desvio-padrao periodo tardio	6	Variabilidade no periodo tardio
Media da variacao ano-a-ano (YoY)	6	Mudancas abruptas - sinal de construcao/demolicao
Desvio-padrao da variacao ano-a-ano	6	Instabilidade da serie (mais alta em Classes 3 e 4)
Indicadores de cobertura temporal (has_early_data, has_late_data)	2	Sinaliza pixels sem observacoes validas num dos periodos

Total: 60 atributos por pixel (12 gerais + 10 de indices + 12 periodo inicial + 12 periodo tardio + 12 YoY + 2 indicadores).

4.3 - Sobre a estrutura do codigo (visao geral dos 4 notebooks)

Notebook	Papel no pipeline	Principais funcoes/objetos
1-Introduction	Contexto do desafio, definicao das classes e regras de submissao	-
2-Reading_Data	Carrega os parquets, deriva age_class, explora colunas, distribuicao, qualidade, correlacoes	assign_age_class(), make_flat_view(), plot_*()
3-Preprocessing	Achata, preenche lacunas, calcula indices, agrega por pixel (60 features), padroniza	fill_time_series_gaps(), compute_spectral_indices(), build_pixe
4-Modelling	Treina e valida o modelo em Madrid; zero-shot e few-shot (prototipos) em Amsterdam	RandomForestClassifier, RepeatedStratifiedKFold, prototype_

Pagina 5 - Processamento e Modelagem: Passo a Passo do Codigo

5.1 - Pre-processamento (Notebook 3) em detalhe

* **Atribuir classe:** assign_age_class() aplica pd.cut sobre weighted_mean_year com as fronteiras especificas por cidade (ver pagina 3).

* **Achatar observacoes:** make_flat_view() seleciona a 1a observacao qa_valid por banda e remove Blue > 15.000.

* **Registar cobertura temporal:** period_coverage() marca, ANTES do preenchimento de lacunas, se o pixel tem alguma observacao real no periodo inicial (<=2003) e/ou tardio (>=2004) - isto preserva um sinal genuino sobre a completude dos dados brutos.

* **Preencher lacunas (gap-filling):** fill_time_series_gaps() cria a grelha completa pixel x ano e usa interpolacao linear (interior) + bfill/ffill (bordas), garantindo uma serie de 40 anos completa por pixel.

* **Calcular indices espectrais:** compute_spectral_indices() adiciona NDVI, NDBI, UI, MNDWI, BSI a cada linha pixel-ano (formulas na pagina 6).

* **Agregar por pixel:** build_pixel_features() faz groupby(pixel) e calcula mean/std gerais, por periodo (early/late) e das diferencas ano-a-ano (YoY), produzindo as 60 features finais.

* **Padronizar:** StandardScaler ajustado (fit) apenas em X_madrid, depois aplicado (transform) a X_madrid e X_amsterdam - protocolo correto para evitar vazamento entre cidades.

* **Serializar:** Os arrays finais (X, y, scaler, feature_names, pixel_ids) sao guardados em preprocessed_data.pkl para o Notebook 4 carregar diretamente.

5.2 - As 5 formulas dos indices espectrais

Indice	Formula	O que capta
NDVI	(NIR - Red) / (NIR + Red)	Vegetacao - valores baixos indicam superficies construidas
NDBI	(SWIR1 - NIR) / (SWIR1 + NIR)	Intensidade de area construida
UI	(SWIR2 - NIR) / (SWIR2 + NIR)	Areas urbanas
MNDWI	(Green - SWIR1) / (Green + SWIR1)	Agua; negativo para solo construido
BSI	((SWIR1+Red) - (NIR+Blue)) / ((SWIR1+Red) + (NIR+Blue))	Solo exposto vs. vegetado

5.3 - Modelagem (Notebook 4) em detalhe

* **Validacao cruzada em Madrid:** RepeatedStratifiedKFold(n_splits=5, n_repeats=5) -> 25 ajustes de modelo no total. Cada ajuste treina um RandomForestClassifier novo e avalia macro-F1 no fold retido (out-of-fold).

* **Modelo final Madrid:** Depois de validada a performance, treina-se um RandomForestClassifier final com TODOS os dados de Madrid (e este modelo que e transferido para Amsterdam).

* **Importancia de atributos:** model.feature_importances_ identifica quais das 60 features mais contribuem para a divisao das arvores (ver pagina 7).

* **Transferencia zero-shot:** O modelo final de Madrid e aplicado diretamente a X_amsterdam (ja padronizado com o scaler de Madrid), sem qualquer rotulo de Amsterdam.

* **Transferencia few-shot (prototipos):** Para cada classe, calcula-se o vetor medio (prototipo) de uma pequena amostra rotulada de Amsterdam (n=5,10,25,50,100,200/classe); os pixels restantes (query) sao classificados pela distancia euclidiana minima ao prototipo mais proximo. Repetido 10 vezes por orcamento de rotulos para estimar variancia.

* **Mapas de incerteza espacial:** predict_proba() e usado para calcular entropia de Shannon e divergencia KL da classe verdadeira, mapeadas geograficamente (px_key, py_key) para visualizar onde o modelo esta confiante ou inseguro.

Pagina 6 - Modelos Integrados: Como se Chegou ao Modelo Final

6.1 - Filosofia geral: Aprendizagem Metrica (Metric Learning)

O notebook adota explicitamente uma abordagem de **metric learning**: em vez de treinar um classificador diferente do zero para cada cidade, a ideia e aprender/usar uma **representacao de atributos** (o vetor de 60 features padronizadas) na qual edificios da mesma idade - independentemente da cidade - ficam proximos no espaco vetorial. Isso permite reaproveitar o modelo de Madrid e adapta-lo a Amsterdam com muito poucos exemplos rotulados locais.

6.2 - Dois modelos combinados no pipeline

Estagio	Modelo	Papel	Dados de treino
1 - Base (Madrid)	RandomForestClassifier (n_estimat[illegible]	[illegible]	[illegible] rotulado d
2 - Transferencia (Ams[illegible]	[illegible]ificador de prototipo (prototype_[illegible]	[illegible]	[illegible]

Estes dois modelos nao sao combinados por ensemble ponderado (nao ha soma de probabilidades entre RF e prototipo neste notebook) - a Random Forest e usada apenas no **modo zero-shot** (baseline sem adaptacao), e o classificador de prototipo e o mecanismo de **adaptacao few-shot**, operando diretamente sobre o mesmo espaco de atributos de 60 dimensoes que a Random Forest usou para aprender em Madrid.

6.3 - Como o classificador de prototipo funciona (a peca-chave da transferencia)

1. Para cada uma das 4 classes, calcular o **prototipo** = media (centroide) dos vetores de atributos (60-D, ja padronizados) dos exemplos rotulados de Amsterdam disponiveis para aquela classe.
2. Para cada pixel nao rotulado ("query"), calcular a distancia euclidiana a cada um dos 4 prototipos.
3. Atribuir a classe do prototipo mais proximo (equivalente a um 1-vizinho-mais-proximo com um "representante" suavizado por media, em vez de um unico ponto - o que reduz o ruido quando ha poucos exemplos por classe).
4. Para mapas de incerteza, converte-se a distancia negativa em probabilidade via softmax (prototype_predict_proba).

6.4 - Por que esta escolha de modelo (justificativa)

- **Random Forest** foi escolhida como baseline robusta: nao exige forte ajuste de hiperparametros, lida bem com atributos correlacionados (bandas espectrais sao altamente correlacionadas) e fornece importancia de atributos nativa. O proprio notebook nota que e usada "como baseline, sem otimizacao de parametros" - e ponto de partida, nao o resultado final ideal.
- **Classificador de prototipos** foi escolhido para a transferencia porque generaliza melhor que k-NN bruto quando ha muito poucos exemplos (5-25 por classe): calcular uma media por classe suaviza o ruido de amostras individuais, tornando a fronteira de decisao mais estavel mesmo com dados escassos.
- O StandardScaler e ajustado **apenas em Madrid** e reaplicado (nao reajustado) em Amsterdam em ambas as etapas - pre-processamento e modelagem -, garantindo que a comparacao entre cidades seja justa e sem vazamento de informacao estatistica de Amsterdam para o modelo.

> *O proprio material do desafio deixa claro que este e um **baseline de referencia**: "Random Forest e usada como baseline, sem otimizacao de parametros - a sua tarefa e melhorar este baseline. Seja criativo, experimente outras tecnicas." Ou seja, o modelo aqui descrito e o ponto de partida oficial do hackathon, nao a solucao definitiva a ser entregue.*

Pagina 7 - Precisao (Desempenho) Obtida pelo Modelo Baseline

7.1 - Metrica principal: macro-F1

A metrica escolhida pelos organizadores e o **F1 macro** (media nao-ponderada do F1 das 4 classes), porque trata todas as classes com igual importancia independentemente do seu tamanho - essencial dado o forte desequilibrio de classes ja identificado.

7.2 - Resultado em Madrid (validacao cruzada, 5-fold x 5 repeticoes = 25 ajustes)

Metrica	Valor
Macro-F1 medio	0,6179
Desvio-padrao entre execucoes	+/- 0,0043
N total de ajustes de modelo (folds x repeticoes)	25

A matriz de confusao fora-da-dobra (out-of-fold) mostra que as **Classes 2 e 4 obtem maior recall**; a **Classe 2 e sistematicamente sobre-prevista** (a maioria dos casos da Classe 1 e confundida com Classe 2) - um sintoma claro de vies causado pelo desequilibrio de classes (Classe 2 e a maior classe em Madrid, 35,6% dos pixels).

7.3 - Resultado em Amsterdam: Zero-Shot (sem qualquer rotulo local)

Classe	Precisao	Recall	F1	Suporte
1 (Pre-1945)	0,4279	0,1016	0,1642	7.568
2 (1945-1984)	0,3661	0,8804	0,5171	8.802
3 (1984-2004)	0,7527	0,1051	0,1845	6.745
4 (2004-2024)	0,6007	0,4355	0,5049	2.877
Macro-F1 = 0,3427 | Accuracy geral = 40,3%				

Ha uma **queda acentuada** de 0,618 (Madrid) para 0,343 (Amsterdam zero-shot) - evidencia clara de "domain gap" entre as duas cidades. A Classe 2 e fortemente sobre-prevista (recall de 88%), confirmando o vies observado tambem em Madrid.

7.4 - Resultado em Amsterdam: Few-Shot com Prototipos (media +/- desvio-padrao de 10 execucoes)

Amostras por classe	Total rotulado	Macro-F1 medio	Desvio-padrao
5	20	0,5437	+/- 0,0324
10	40	0,5649	+/- 0,0312
25	100	0,5986	+/- 0,0176
50	200	0,6079	+/- 0,0101
100	400	0,6129	+/- 0,0097
200	800	0,6150	+/- 0,0036

Mesmo com apenas **5 amostras por classe**, o F1 ja salta de 0,343 (zero-shot) para 0,544 - um ganho enorme por um custo minimo de rotulagem. Com **100-200 amostras/classe**, a performance em Amsterdam (=0,61-0,615) praticamente converge para o nivel obtido em Madrid por validacao cruzada (0,618), fechando quase totalmente o "gap" entre cidades.

7.5 - Atributos mais importantes (Random Forest, mean decrease in impurity)

#	Atributo	Importancia
1	NIR_early_mean	0,0387
2	Red_late_std	0,0297
3	NIR_mean	0,0283

4	UI_mean	0,0264
5	Red_std	0,0256

A importancia esta bastante distribuida entre as 60 features (nenhuma domina isoladamente), o que reflete a alta correlacao entre bandas espectrais - mas tambem sugere que o modelo usa combinacoes complexas de atributos, dificeis de intuir manualmente.

Pagina 8 - Resultados e Saidas do Modelo

8.1 - O que o modelo produz, exatamente

Importante esclarecer: *o modelo NAO preve a idade media em anos, nem um valor continuo de "weighted_mean_year". A saida (y previsto) e sempre um* ***rotulo categorico discreto de 1 a 4*** *- a classe de idade a que o pixel provavelmente pertence. A coluna continua weighted_mean_year existe apenas nos dados de treino, como base para GERAR o rotulo - ela nunca e o alvo da regressao do modelo, porque este e um problema de classificacao, nao de regressao.*

Cada classe representa uma **faixa (bracket) de idade**, nao um unico valor medio - mas e possivel, se necessario para relatorio, converter a classe prevista numa "idade estimada representativa" usando o ponto medio do intervalo correspondente (ex.: Classe 3 em ambas as cidades -> ponto medio =1994; Classe 1 em Madrid, sem limite inferior definido nos dados, exigiria assumir um limite razoavel, ex. 1850, para calcular um ponto medio aproximado). Isto seria um passo extra de pos-processamento, nao parte do pipeline atual.

8.2 - Formato exato da saida por pixel

Campo	Exemplo	Origem
px_key, py_key	14194, 149671	Identifica a localizacao geografica do pixel
age_class previsto	3	model.predict(X) - Random Forest (Madrid) ou classificador de prototip
Probabilidades por classe (opcional)	(0.05, 0.10, 0.70, 0.15)	model.predict_proba(X) - usado para mapas de incerteza
Classe de idade correspondente (rotulo textual)	'1984-2004'	Mapeamento via CLASS_LABELS_MADRID / CLASS_LABELS_AMST

8.3 - Saidas visuais e analiticas geradas pelo pipeline

- **Matrizes de confusao** (normalizadas por linha = recall) para Madrid (CV), Amsterdam zero-shot e Amsterdam few-shot (25 e 100 amostras/classe).
- **Mapas espaciais** reconstruidos a partir de (px_key, py_key): classe verdadeira vs. prevista, acerto/erro por pixel, e mapas de incerteza (entropia de Shannon e divergencia KL da classe verdadeira).
- **Grafico de importancia de atributos** (top 20 features da Random Forest).
- **Curva de desempenho vs. orcamento de rotulos**: macro-F1 (+/- desvio-padrao) em funcao do numero de amostras Amsterdam por classe, em escala logaritmica - o grafico-chave exigido na entrega final.

Um achado visual relevante: mesmo com accuracy moderada, os mapas espaciais mostram que o modelo identifica corretamente as **grandes zonas urbanas** dominadas por cada classe - sugerindo que um pos-processamento espacial (ex.: suavizacao, votacao por vizinhanca) sobre as probabilidades poderia melhorar ainda mais a precisao final.

Pagina 9 - Como Aplicar o Modelo a Amsterdam (Passo a Passo Pratico)

Esta seccao resume o procedimento operacional completo para adaptar o modelo treinado em Madrid a Amsterdam, reunindo o que foi feito nos Notebooks 3 e 4.

Passo 1 - Preparar os dados de Amsterdam da mesma forma que Madrid

Aplicar exatamente o mesmo pipeline do Notebook 3 aos dados de Amsterdam: derivar age_class (com a fronteira 1945, especifica de Amsterdam), achatar observacoes, remover outliers de Blue > 15.000, preencher lacunas temporais, calcular os 5 indices espectrais e agregar para as 60 features por pixel.

Passo 2 - Padronizar SEM reajustar o scaler

Aplicar X_amsterdam_std = scaler.transform(X_amsterdam) usando o MESMO StandardScaler ja ajustado em Madrid (nunca chamar fit() novamente em Amsterdam) - isto e essencial para manter o mesmo espaco de atributos onde o modelo de Madrid "faz sentido".

Passo 3 (opcional/baseline) - Rodar zero-shot

Antes de rotular qualquer coisa em Amsterdam, aplicar diretamente model.predict(X_amsterdam_std) com o Random Forest de Madrid. Isto da uma referencia inferior (macro-F1 =0,34) e serve de comparacao para medir o ganho da adaptacao few-shot.

Passo 4 - Obter uma pequena amostra rotulada de Amsterdam

Selecionar aleatoriamente N pixels por classe (N em {5, 10, 25, 50, 100, 200}, conforme o orcamento de rotulagem/tempo disponivel) cujo weighted_mean_year real seja conhecido - estes formam o "support set".

Passo 5 - Calcular os prototipos de classe

Para cada uma das 4 classes, calcular o vetor medio (media elemento-a-elemento das 60 features padronizadas) dos pixels rotulados daquela classe no support set. Resultado: 4 vetores de 60 dimensoes.

Passo 6 - Classificar o restante de Amsterdam por prototipo mais proximo

Para cada pixel nao incluido no support set ("query set"), calcular a distancia euclidiana aos 4 prototipos e atribuir a classe do prototipo mais proximo - funcao prototype_predict().

Passo 7 - Validar com repeticoes aleatorias

Repetir os Passos 4-6 multiplas vezes (ex.: 10 vezes) com amostras diferentes do support set, para obter media e desvio-padrao do macro-F1 - essencial porque amostras pequenas tem alta variancia.

Passo 8 - Escolher o orcamento de rotulos adequado

Usar a tabela da pagina 7 (seccao 7.4) para decidir quantos pixels rotular: 25/classe ja recupera boa parte do desempenho (F1=0,60) com esforco minimo de anotacao (premio "low-data"); 100-200/classe aproxima-se do teto de Madrid (F1=0,61-0,615), mas exige mais esforco de rotulagem.

Passo 9 - Gerar mapas e relatorio final

Usar predict_proba (via softmax de distancias negativas) para gerar mapas de incerteza espacial e a matriz de confusao final de Amsterdam, e reunir os quatro F1 (5/25/50/100/200 por classe, conforme pedido nos criterios de avaliacao) na tabela e no grafico F1 vs. log2(n amostras) exigidos na entrega.

Ponto critico a nao esquecer: *o modelo de Random Forest treinado em Madrid NAO e re-treinado com os dados de Amsterdam. A adaptacao acontece inteiramente no classificador de prototipos, que opera no mesmo espaco de atributos padronizados. Isso e o que torna a abordagem "transferivel" e replicavel para outras cidades alem de Amsterdam - bastando repetir os Passos 1 a 9 com poucos rotulos locais de qualquer nova cidade.*